

设计测试用例（Test Case Design）是软件测试中最核心的工作。它的目标是：**用最少的测试步骤，发现尽可能多的 Bug。**

设计测试用例不仅仅是“把需求抄一遍”，而是需要运用特定的逻辑和方法来覆盖各种边界和异常情况。

以下是设计测试用例的完整指南，包括**结构、核心设计方法和实战流程。**

一、 测试用例的标准结构

在 Excel 或测试管理工具（如 Jira/TestLink/禅道）中，一个标准的测试用例通常包含以下字段：

字段名	说明	示例
用例编号 (ID)	唯一标识，方便追踪	TC-LOGIN-001
标题 (Title)	简明扼要地描述测试点	验证用户使用正确的用户名和密码能成功登录
前置条件 (Pre-condition)	执行前必须满足的状态	用户已注册，且处于未登录状态
测试数据 (Test Data)	具体输入的数据	用户名：admin，密码：123456
执行步骤 (Steps)	详细的操作指导	1. 打开登录页 2. 输入用户名 3. 输入密码 4. 点击登录按钮
预期结果 (Expected Result)	系统应该给出的反应	跳转至首页，且右上角显示“欢迎 admin”
实际结果 (Actual Result)	执行测试时填写	(测试时填写，如：提示密码错误)
状态 (Status)	执行测试时填写	Pass / Fail / Block

二、 核心设计方法 (黑盒测试五大金刚)

这是设计用例的“灵魂”。你不能只测“正常情况”，必须利用以下方法挖掘隐藏的 Bug。

1. 等价类划分法 (Equivalence Partitioning)

核心思想：把无限的输入数据分成若干个“类”，每个类里选一个代表数据测就行了。

- **有效等价类：**符合需求的数据（如：输入 1~100 之间的整数，选 50）。
- **无效等价类：**不符合需求的数据（如：输入非整数 abc，或超范围的 200）。
- **目的：**减少测试用例数量，避免穷举。

2. 边界值分析法 (Boundary Value Analysis) —— 最容易发现 Bug 的方法

核心思想：大量的错误发生在输入范围的**边缘**，而不是中间。

- **规则：**如果需求规定范围是 1 到 100。
- **必测点：**
 - **上点**（边界上的点）：1, 100

- **离点**（离边界最近的点）：**0, 101** (如果是整数)
- **内点**（范围内的点）：**50**

3. 判定表法 (Decision Table)

核心思想：适用于**多种条件组合**的情况。

- **场景：**比如“订单折扣逻辑”——如果是VIP **且** 满500元 **且** 在生日当天，打5折；否则打8折。
- **做法：**列出所有条件的 True/False 组合，确保每种组合都有对应的用例。

4. 场景法 (Scenario Testing) / 流程分析法

核心思想：基于业务流程（Use Case / User Story）来设计。

- **基本流 (Happy Path)：**模拟用户一步不错地走完流程。
- **备选流 (Sad Path)：**模拟用户中途取消、支付失败、网络断开等异常流程。

5. 错误推测法 (Error Guessing)

核心思想：基于测试人员的经验和直觉。

- **做法：**比如测试搜索框时，故意输入空格、特殊字符（%、'）、超长字符串、SQL注入代码等，看系统是否崩溃。

三、 实战演练：以“用户登录”为例

假设需求如下：

用户名：长度 6-10 位。密码：必填。失败限制：连续失败 3 次锁定账号。

设计步骤与用例产出：

ID	测试场景	设计方法	输入数据	预期结果
TC-01	正常登录	场景法/等价类	User: abcdef (6位) Pass: 123456	登录成功
TC-02	用户名长度边界(最小)	边界值	User: abcdef (6位)	登录成功
TC-03	用户名长度边界(最小-1)	边界值	User: abcde (5位)	提示“长度需6-10位”
TC-04	用户名长度边界(最大)	边界值	User: abcdefghij (10位)	登录成功
TC-05	用户名长度边界(最大+1)	边界值	User: abcdefghijk (11位)	提示“长度需6-10位”
TC-06	密码为空	错误推测/等价类	User: validUser Pass: [空]	提示“密码必填”

ID	测试场景	设计方法	输入数据	预期结果
TC-07	特殊字符处理	错误推测	User: admin' or '1'='1 Pass: any	提示“用户名或密码错误”（防SQL注入）
TC-08	锁定机制(逻辑)	判定表/状态	连续输入错误密码 3 次	第3次后，账号被锁定，无法再尝试

四、从需求到用例的思维导图

当你拿到一个用户故事或用例规约时，按这个步骤思考：

- 1. 第一层：界面检查 (UI)
 - 按钮位置对吗？文字有错别字吗？Tab键顺序对吗？
- 2. 第二层：功能验证 (Function)
 - 正向：输入正确数据，能保存/跳转吗？（覆盖基本流）
 - 逆向：输入错误数据，报错提示友好吗？（覆盖等价类、边界值）
- 3. 第三层：数据与逻辑 (Data & Logic)
 - 数据库存的数据对吗？
 - 状态流转对吗？（如：未支付->已支付）
- 4. 第四层：非功能需求 (Non-functional)
 - 性能：点登录要等多久？
 - 兼容性：在 Chrome 和 Edge 上都能用吗？手机上能用吗？
 - 安全性：密码是明文传输的吗？

五、什么样的用例才是“好用例”？

- 1. 覆盖率高：不仅仅测“能跑通”，还要测“跑不通”和“边界”。
- 2. 无二义性：步骤写得谁都能看懂。不要写“输入适当的数据”，要写“输入数字 10”。
- 3. 独立性：TC-02 的执行最好不依赖 TC-01 的结果（虽然很难完全做到，但要尽量解耦）。
- 4. 可维护性：如果需求变了，用例容易修改。

总结

设计测试用例就是***有组织地找茬”。先用场景法覆盖业务流程，再用边界值和等价类去抠输入框的细节，最后用错误推测**去攻击系统的弱点。